



摘要：

Hermes Agent爆火的核心不在功能碾压，而在“替用户做决定”的极致自动化：自动生成并进化Skill、主动高频记忆写入、全局用户建模，将复杂度从用户端转移到底层规则系统。同时引入 GEPA 算法实现技能自进化，配合硬编码安全与上下文管理，优先保证稳定性。尽管在复杂任务中仍易失控，但在高容错场景已形成明显体验优势，并率先占据“全自动 Agent”心智高地。

进入 2026 年 4 月，才火了两个月的 OpenClaw（俗称“龙虾”）就迎来了它的挑战者。Hermes Agent 连续数周占据 GitHub Trending 榜首，狂揽 22,000 颗星。

它火到什么程度呢？连 Anthropic 都要抄它的。4 月 10 日，Nous Research 的创始人 Teknium 跳出来吐槽，说 Anthropic 正在「复制」Hermes 自动判断任务完成、主动提醒用户的功能。社区叙事也因此非常统一，认为 Hermes 凭借自进化 Agent、自动记忆管理和用户建模系统，在技术上全面超越了前任王者 OpenClaw，重新定义了开源 Agent 的方向。

不过，如果抛开这些宏大叙事，真正把两边拆开对比，你会发现它们在功能上一模一样的地方，远比差异多得多。

比如定时调度，两边都有。Hermes 支持人类可读的格式和标准的 cron 表达式，每个任务都跑在隔离的会话（session）里。OpenClaw 也同样支持 at、every、cron 三种调度类型，任务直接持久化写进本地的 JSON 文件，重启也不会丢。

再比如子 Agent 委派，两边都有。Hermes 的 delegate_task 支持单任务和最多 3 个并行子任务，子 Agent 环境完全隔离，干完活只返回一个摘要。OpenClaw 的 sub-agent 机制也支持这种后台隔离执行和结果回传，甚至还能配置嵌套深度。

浏览器自动化、TTS（语音合成）、Vision 视觉能力、图像生成、语音交互，两边也全都有。Gateway 方面，Telegram、Discord、Slack、WhatsApp、Signal 等 20 多个平台的消息集成，两边也毫无悬念地全都有。

对着清单一项项打勾就会发现，两者的功能几乎完全重合。所谓功能表上的「绝对碾压」根本不存在。

那么问题来了，既然功能都一样，Hermes 凭什么火成这样？社区里被吹上天的「自进化」「自动记忆」「用户建模」，到底有多少是真正的底层结构差异？



Hermes		OpenClaw	
定时调度	✓	定时调度	✓
子Agent	✓	子Agent	✓
浏览器	✓	浏览器	✓
语音合成	✓	语音合成	✓
视觉能力	✓	视觉能力	✓
消息集成	✓	消息集成	✓
功能几乎完全重合			

01 会自己长大的 Skill

翻遍两边的默认配置，你能找到的唯一硬核差异就一个，即，Hermes 在 Skill（技能）上实现了自动进化的闭环。

Skill 算是 Agent 的工作流知识单元，说白了就是一个 Markdown 文件，专门告诉 Agent 遇到某类任务该按什么步骤干、中间调什么工具、搞砸了怎么救场。

Hermes 把技能的生命周期硬生生劈成了两截。一截是运行时的静默生成，另一截是离线的硬核进化。

先说生成。平时让 Agent 干活，只要它在中间调用了 5 次以上工具，或者出了错又自己把流程救回来了，又或者你作为用户直接纠正了它的输出，主仓库里一套写死的硬规则就会被触发。Agent 会默默把刚才那套跑通的工作流打个包，存成本地的 SKILL 文件。这一步完全静默，很多时候你根本不知道它又给自己写了个新技能。

等下次再遇到类似任务，它会自动去扫索引。这个加载过程分四层渐进，就像去图书馆找资料。它先看目录卡片（Tier 0），只把名称和描述塞进系统提示词里，大概占 3,000 个 token。方向对了，再逐层去书架拿书，把完整内容展开。

但真正让 Hermes 拉开身位的，是第二步的进化。

Hermes 内置了一套离线批量进化算法，还专门拉了一个独立仓库（hermes-agent-self-evolution）。引擎用的是 DSPy 框架，加上一套叫 GEPA 的核心算法。

GEPA 的全称是 Genetic-Pareto Prompt Evolution。这套体系并非 Hermes 自创，出自 Lakshya Agrawal 等人的一篇 ICLR 2026 Oral 论文，标题就叫《反思性提示词进化可以跑赢强化学习》。

现在的学术圈搞技能进化，大部分都在走 RL（强化学习）的路线。像 SkillIRL 或者 SAGE 这些框架，连名字都带着 RL，指望用梯度更新来强化技能库。但 GEPA 走了一条完全对立的路，刻意抛弃了强化学习。GEPA 论文本身就是在证明一件事，哪怕没有梯度更新，靠大模型的反思能力加上进化算法，不仅能跑赢 RL，样本利用效率还更高。

它是怎么做到的呢？这套算法有三个硬核的底座。

首先是反思性变异（Reflective mutation）。它不是瞎猜式的随机变异。大模型会去读之前的执行轨迹（trace），自己反思这次为什么做对了，为什么做错了，提示词到底该改哪几个字。



其次是帕累托前沿选择（Pareto frontier selection）。生成了一批变异的候选技能后，它不是一刀切只留全局均分最高的。只要某个候选在哪怕一个评估样本上表现最强，它就会被保留下来。这么做是为了保证技能探索的多样性和鲁棒性。

最后是自然语言反馈作为变异信号。传统 RL 靠数值 reward 引导参数更新，但数值信号颗粒度太粗，跑了一次得 0.6 分，你根本不知道是哪里对哪里错。GEPA 的每次变异用的都是具体的自然语言反馈，比如「这一步没检查边界条件」「应该先读配置再写缓存」。LLM 读得懂这种反馈并据此产生下一轮变体，比解读一个浮点数有效得多。

把它串成工作流就是这样。系统定期去读现有的 SKILL 文件，去历史会话里抽样（或者干脆自己合成）搞出一个评估集。然后 GEPA 介入，看执行轨迹，反思提意见，生成候选变体，跑一轮评估，最后用帕累托算法挑出赢家。

这套离线的进化闭环跑完，得出优化后的 Skill 后，它不会直接覆盖原文件，而是老老实实生成一个 PR（Pull Request），必须要等你作为人类审核员点头合并，这个进化的技能才会真正生效。系统永远不会进行直接提交。

这直接戳破了社区里那套「用户完全无需介入」的爽文神话。Hermes 的态度其实很明确，技能生成可以全自动且静默，但技能进化必须过人眼。

回过头看看 OpenClaw。它也不是没有 Skill 系统，但要命的是每一步都得靠你主动。你需要手动建文件、手动安装、再手动授权，三个条件凑齐了技能才会生效。搞了个新 Skill 还得重启它统一管理的 Gateway 网关进程，系统才能认得出。

而且它的加载极其简单粗暴，根本不做任务匹配，只要配了就全量塞进上下文里，除非你手动加个禁用标签把它踢出去。

两边都有 Skill。真正的区别就在于谁来按下启动键。Hermes 说「放着我来」，OpenClaw 说「你自己搞」。

02 谁在替谁记事

如果说 Skill 解释了 Hermes 为什么「越用越快」，那社区里传得神乎其神的另一半叙事「它懂我是谁」，就得归功于记忆系统了。

现在的三大主流开源 Agent（Claude Code、OpenClaw、Hermes）其实都有自动记忆。但只要稍微深挖就会发现，它们服务的对象、触发机制和记忆保质期完全是两码事。

先说 Claude Code。它的自动记忆（auto-memory）是默认开着的，平时干活时会自动把构建命令、调试经验、架构笔记甚至代码风格都记下来，而且每 24 小时就跑一次 Auto Dream 来整理，把过期或者自相矛盾的东西清掉。听起来很智能，但这套系统有着极其严格的项目隔离。

它的边界卡死在 git root（项目根目录）上，项目 A 里学到的血泪教训，绝对带不到项目 B 里。它不记你的个人偏好，不关心坐在屏幕对面的是谁，脑子里只有「这个项目该怎么跑」。

再说 OpenClaw，它的记忆系统就更偏长程。每次启动对话，它都会把包括 MEMORY.md 和 USER.md 在内的 8 个底层文件强制灌进自己的脑子里。这两个文件不仅跨项目共享，而且会自动写入。



那它是如何写入的呢？它的写入机制极其被动，更像兜底。在每次对话的上下文（token）快要撑爆、系统准备做大压缩（compaction）之前，Agent 会悄悄跑一个 silent turn（隐藏轮次）。它会在本轮次里，把当前聊过的重点随手记到当天的日记文件里，同时把关于你的偏好写入长期挂载的 MEMORY.md 或 USER.md。

所以你很久没用 OpenClaw，隔几天一打开发现它「居然还记得你是谁」，靠的就是这张被动结成的长期大网，那些偏好早就被塞进了几个启动必读的文件里。这确实能让人产生「这 AI 可以养」的实感。但本质上更像是一种求生本能，眼看脑子装不下了赶紧存一下档。至于那些老旧日记，如果不用外挂的语义向量数据库支持，它只能靠关键词生搜。

在这个维度上 Hermes 是另一套逻辑。在 v0.7 版本之前，Honcho 是 Hermes 里唯一写死的长期记忆后端，没有别的选项。

这个之前是默认选项的 Honcho 设计得很巧妙。绝大多数 Agent 的记忆系统（包括 Hermes 的默认内置记忆）本质上是一个被动的记录仪。你聊了什么，它切碎了转成向量塞进数据库，下次遇到相似的话题再通过计算距离（Embedding 余弦相似度）捞出来。

Honcho 不走这条路。它是一个「AI 原生」的记忆后端，主打的是异步的辩证推理（Dialectic reasoning）和深度实体建模。

你跟 Agent 聊完天，主会话结束了，但 Honcho 的活才刚开始。它会在后台拉起额外的模型调用，对刚聊完的历史进行分析，提炼出你话里的概念（Entity），提取底层偏好，甚至把你前后矛盾的话进行辩证和对齐。它把你随口说出的碎碎念，计算成结构化的「洞察（Insight）」。

听着非常先进，但它也非常费 token，容易把关键细节给洗掉。设成插件，更安全。

但就算没有 Honcho，Hermes 的记忆写入都比 OpenClaw 主动得多。Hermes 搞了个微调（nudge）机制，根本不等脑子撑爆，大概每聊 15 轮对话就会被硬性触发一次。这就是系统强制塞给 Agent 的一条反思指令，赶紧回顾一下刚才聊的，看看这人有什么习惯值得记一笔。这种高频的主动反思，让 Hermes 在同时间里写进持久文件的信息量大得惊人。

不仅写入更积极，Hermes 找回记忆的手法也更硬。它在默认架构里内置了 SQLite FTS5 的全文检索能力。不用再去费劲配什么词向量服务，Agent 想翻旧账，直接就能去庞大的过往聊天记录里扫街。

把这三家摆在一起看，那条进化线就清晰了。OpenClaw 是一套被动触发的长期记忆体系。Claude Code 做到了主动记录和整理但底线是对事不对人。而 Hermes 把触发时机做得极其主动，记忆插件随意切，全局共享，还默认配齐了能翻遍所有历史的检索利器。

日常用起来的体感差异也就是这么拉开的。OpenClaw 是在快崩溃前才想起来认识你一下。Hermes 则是每隔一会儿就在暗中揣摩你的心思，并且能随时翻出你们俩说过的话。



03 把复杂度藏起来

无论是 Skill 的自生成，还是记忆的高频主动写入，背后指的其实都是同一件事，即，Hermes 只是替你把本该你做的决定都做了。

但是系统复杂度这种东西是守恒的。

你不用动手，不代表决策凭空消失了，它只是从你的手动操作转移到了底层硬编码的死规则里。

在搭这套 harness (Agent 外壳) 的过程里，Hermes 的设计者领悟了一个道理，模型判断不可信，那就做成死规则。

这套 harness 远比 Anthropic 之类的要死。Agent 干活的时候并不是一个纯粹的大模型在裸跑思考，大模型外面严严实实地包着一层代码框架，这层框架里写满了条件判断。

工具调用满 5 次了吗？对话轮数凑够 15 轮了吗？刚才是不是刚死里逃生重试了一次？用户有没有明确开口指出错误？这些问题系统根本打算交给大模型去模糊判断，而是用确定性的代码一条条死盯。条件一满足，立刻执行写好的动作，去生成初始技能，或者硬塞反思指令，又或者把某句话记进长期文件里。

这些遍布各处的防御网，就是被转移走的那部分复杂度。本来该由用户在使用过程中自我规范，现在全写死在 Hermes 的代码里。

而 Hermes 写这些规则依靠的就是设计判断。调用 5 次工具触发技能生成，设成 3 次太容易误触发，设成 8 次又可能漏掉有价值的工作流。每 15 轮反思一次，而不是每轮都反思，因为那会产生海量垃圾记忆且烧钱。

你坐在屏幕前觉得什么都不用管真爽，背后是 Hermes 的开发团队提前把所有判断逻辑替你写死了。

自动化并没有消灭决策，它只是把决策藏到了看不见的地方。

为了保证这套硬规则在没有人类盯着的时候不翻车，Hermes 在底层做了一系列防御性设计。

首先看上下文管理。当对话撑到 85% 阈值时，Hermes 根本不叫大模型来做智能摘要，它的 ContextCompressor 就是一套纯粹的字符串替换逻辑，把旧的工具输出直接换成一个占位符，粗暴但绝对安全。而记忆层面它用的是冻结快照，开机时把记忆一次性倒进系统提示词里，中



途不刷新，等下次重启才生效。这牺牲了实时性，但换来了前缀缓存稳定的命中率，直接砍掉了大概 75% 的 token 输入成本。两个选择的精神一致，session 内部不让 LLM 去做关于上下文和记忆的动态判断，用最笨的规则保住确定性。

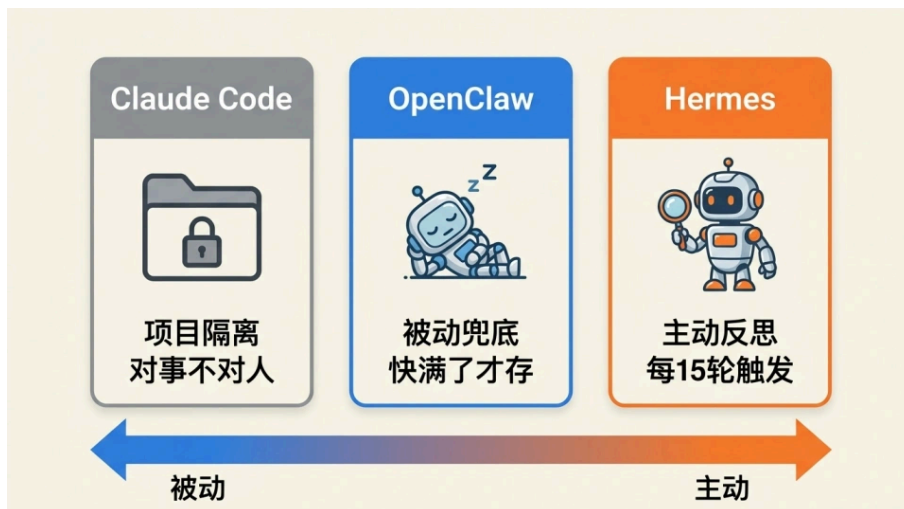
再看它的安全审查。内置的 Smart 审批模式同样不让大模型当裁判去判断一条命令危不危险，而是直接拿一套硬编码的黑名单去正则匹配终端操作。匹配中就必须人类点头确认。

它甚至连搞生态扩展的插件系统，都把开发者当成了潜在敌人。Event Hooks 系统里有 6 种钩子，其中 5 种全都是触发即忘（fire-and-forget）的看客，系统根本不管它们的返回值。想修改 Agent 的运行上下文只有一个唯一的注入点。官方死死卡住一条底线，就算插件代码跑崩了也绝不拖垮 Agent 的主循环。

这几个看似保守的选择，底层逻辑高度一致。

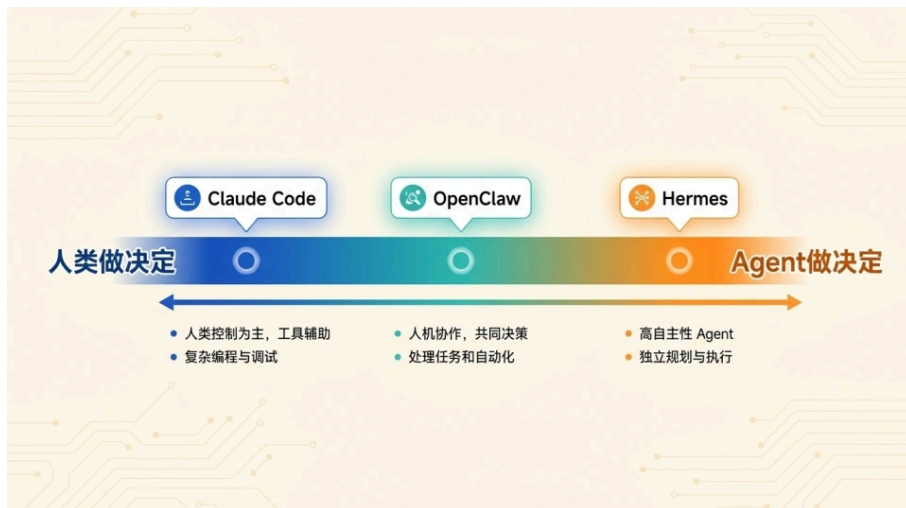
今年初 Chroma 团队做过多轮对话压测，单轮变多轮后，模型平均性能直接掉 39%，最惨能掉 85%。同一时期的记忆综述论文也指出，在超长上下文施压下，死板的规则化跑分愣是碾压了让大模型全权统管的方案。

说白了，当大模型驾驭超长上下文的底子还虚的时候，那些听来越智能的统管方案越容易原地爆炸。反倒是越笨的死规则越靠得住。在平时跑任务的环境里，不翻车才是最高优先级。这是一个诚实的工程判断。



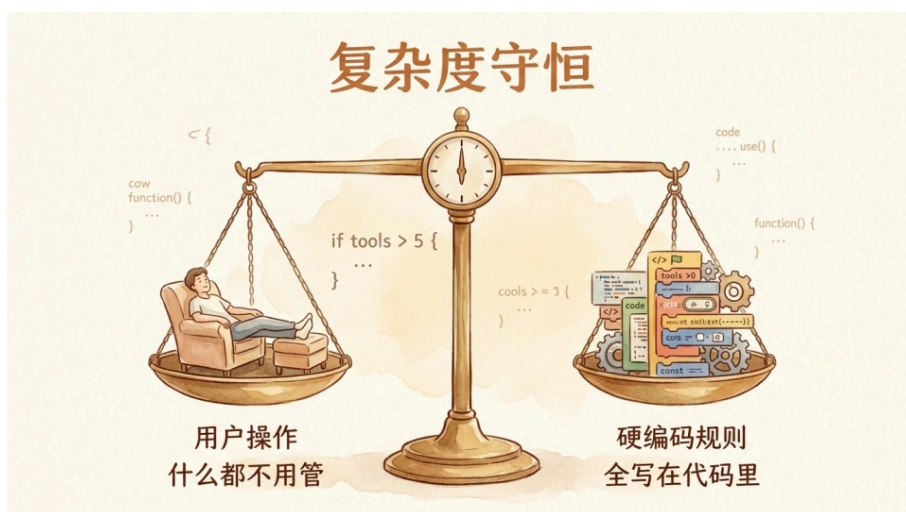
04 一条「干预递减」的光谱

把这几家串起来看，不管是自动写技能、主动存记忆，还是故意退一步的技术栈选择，最终落点都是同一个产品判断，到底该让用户管多少事？



从 Claude Code 和 Codex，到 OpenClaw，再到 Hermes，市面上的开源 Agent 其实铺成了一段清晰的光谱。一端是「所有决定都由人来做」的生产工具，必须让开发者看 diff、批命令、盯每一步。在真实业务里精确控制永远是第一顺位，这不是技术不到位，是产品定位决定的。另一端是「全都交给 Agent 包办」的自动化工具。Hermes 直接站在了最远那头。

它押注的是大部分用户既不想弄懂、也不屑于弄懂 Agent 怎么跑起来。你只管张嘴提需求，技能匹配、记忆分类、上下文压缩，全都在阴影里完成。它的野心不是让你觉得它好用，而是让它在不知不觉中自己越变越好。



05 扛不住硬活，但方向对了

尽管 Hermes 已经很克制，用各种技术保守的规则系统剥夺了模型的自由判定权，以使这个更自动化的系统能够稳定。就放了这么点非规则的权限，模型就开始拉垮。

重灾区就是那套引以为傲的技能系统。不止一个高阶玩家抱怨，自己花好几个小时手动精调出来的技能，被全自动的进化流程直接覆盖掉，这完全是没法接受的灾难。

自动记忆微调的机制同样经不起细看。nudge 的本质是让 Agent 自己判断这轮有没有值得记的，但社区发现 Hermes 判断自己「是否完成了任务」时几乎总觉得自己成功了。所以反思出来的记忆，非常薄弱。

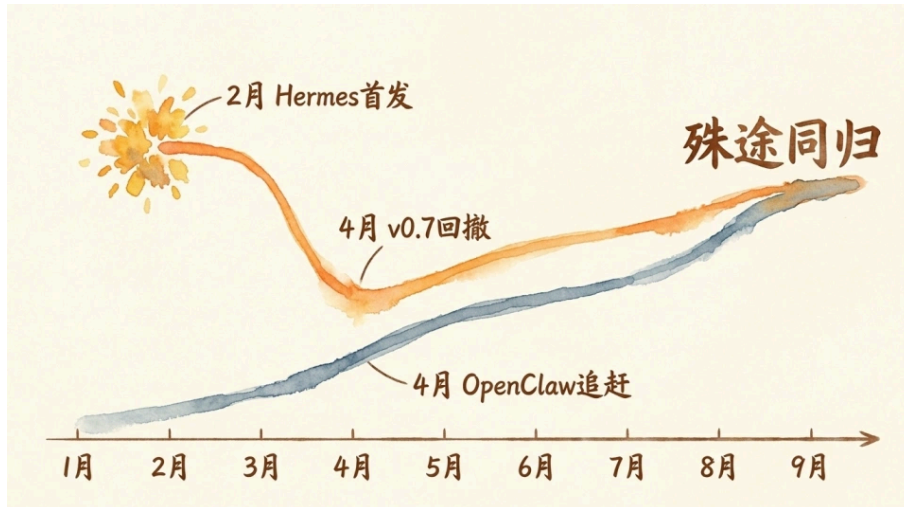
如果是拿去拟一份核心合同、过一遍底层代码、或者搭复杂的财务模型，全自动模式本身就是一个巨大的隐患。这也是那些老牌专业工具不敢直接上全自动的原因，专业人士是为 Agent 的脑抽付出真金白银代价的。

但在写周报、翻天气、或者理一理本地文件这种容错率极高的日常重复任务里，Hermes 现在确实能站得住。只要你给它二十几次迭代的机会，那些慢慢攒起来的技能和记忆，确实能让它跑得又稳又快。

这实打实的体感，足够它圈住第一波忠实用户了。

更重要的是，它踩在了一个基本确定的方向上。

梳理这两个月的发展，你会看到一条非常有意思的演进弧线。



2026 年 2 月 25 日 Hermes 首发，打出的旗号是「与你共同成长的 Agent」。它靠着主动记忆、自动进化和强行替用户做决定的激进路线，一口气冲到了 57,200 颗星。

但到了 4 月 3 日发布的 v0.7 韧性更新里，它其实悄悄往回撤了半步。Hermes 引入了抽象接口，把原本写死在系统里的唯一高级记忆后端 Honcho，降级成了和新接入的 MEMO、ByteRover 等 6 个第三方服务平起平坐。反而把最原始的纯文件加全文检索顶成了默认兜底方案。

一个自称替你决定一切的系统，主动把复杂的方案剥离，把记忆的选择权交还给了用户。这不是硬编码规则派走不下去，毕竟微调触发、技能生成这些核心机制一个没动。这是先行者在撞上社区真实投诉后的一种战略让步，觉得现在的规则系统还吃不透所有复杂场景，有些选择不必强行替用户做。

而另一边的 OpenClaw，动作却是截然相反的补课式加强。过去短短几天里它连跑两步。4 月 5 日放出了类似 Claude Auto Dream 一样的 Dreaming 做离线记忆整理，把已有的短期流水文档在离线时段提炼、评分、晋升为持久的 MEMORY.md 条目。4 月 10 日的更新里，它又砸出 Active Memory，直接在主回复前跑一个专门的记忆子 Agent。这套大模型做裁判的主动派打法，粒度比 Hermes 固定 15 轮一次的微调还要细、还要聪明。

这说明，不管是 Anthropic 还是 OpenClaw，大家全都在往「替你做决定」这条路上靠。Hermes 只不过是下注下得最早，也最狠。

它用两个月的时间死死卡住了全自动 Agent 的叙事高地和用户心智。当 OpenClaw 在后面拼命追赶、用更高级的子 Agent 补齐主动记忆机制的时候，心智已立的 Hermes 反而有了谨慎和退让的余裕。

它赌的不是今天的系统能有多完美，而是在提前做一场局。它算准了只要底层模型的上下文能跨过那条及格线，今天这些硬凑出来的规则安全线就能跟着往上涨，快照可以实时刷新，硬编码条件也能放心交给大模型去判断。那些看似保守的默认层，迟早有一天会彻底吃掉今天显得高大上的接口层。

这叫先占生态位，再等技术升级。等到质变那天，Hermes 早已经攥着真实用户、技能生态和记忆底座站在终点了。

Agent 这片修罗场里，谁先在技术将将够用的时候用兜底工程卡住位置、让产品能用起来，往往比单纯的技术领先管用得多。Manus 是这样，OpenClaw 是这样，如今的 Hermes 也是这样。

本文来源：[腾讯科技](#)

风险提示及免责条款

市场有风险，投资需谨慎。本文不构成个人投资建议，也未考虑到个别用户特殊的投资目标、财务状况或需要。用户应考虑本文中的任何意见、观点或结论是否符合其特定状况。据此投资，责任自负。

限时权益申领

* 体验资格每人限申领一次

扫码
免费领取

VIP会员·7天畅读卡 | 大师课·入门串讲



限免

265 收藏

分享到:

写评论

请发表您的评论

表情 图片

发表评论